



UNIVERSIDADE FEDERAL DE ALAGOAS
INSTITUTO DE COMPUTAÇÃO
GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO
DISCIPLINA: PROGRAMAÇÃO 2

Relatório de Design - Milestone 2

Autor: Davi Vieira Lourenço Correia

Disciplina: Programação 2 (Orientação a Objetos)

Data: 05/05/2025

Introdução

Este relatório apresenta o design do projeto *Jackut*, uma rede social desenvolvida para a disciplina de Programação 2 (Orientação a Objetos). O sistema foi concebido para gerenciar usuários, comunidades, mensagens e diferentes tipos de relações sociais, como amizades, ídolos, interesses românticos e rivalidades. Este documento detalha as escolhas de design, os princípios de orientação a objetos aplicados, a organização do código e os padrões de projeto utilizados, com o objetivo de criar um sistema robusto, manutenível e extensível.

Organização do Projeto

O projeto *Jackut* foi estruturado em pacotes que separam claramente as responsabilidades, promovendo modularidade e organização. O pacote principal, `br.ufal.ic.p2.jackut`, contém as classes centrais do sistema, enquanto o subpacote `br.ufal.ic.p2.jackut.exceptions` abriga exceções personalizadas para tratamento de erros. Entre as classes principais, destaca-se a classe `User`, que gerencia perfis de usuários, mensagens e conexões sociais; a classe `Community`, responsável por interações e mensagens em grupos; e a classe `Message`, que armazena comunicações individuais e coletivas. A classe `Facade` oferece uma interface simplificada para interação com o sistema, seguindo o padrão de projeto `Facade`, enquanto a classe `Intern` coordena a lógica principal. Além disso, as classes `UserManager`, `CommunityManager` e `SessionManager` gerenciam, respectivamente, operações de usuários, comunidades e sessões, e a classe `SystemPersistence` cuida da persistência de dados.

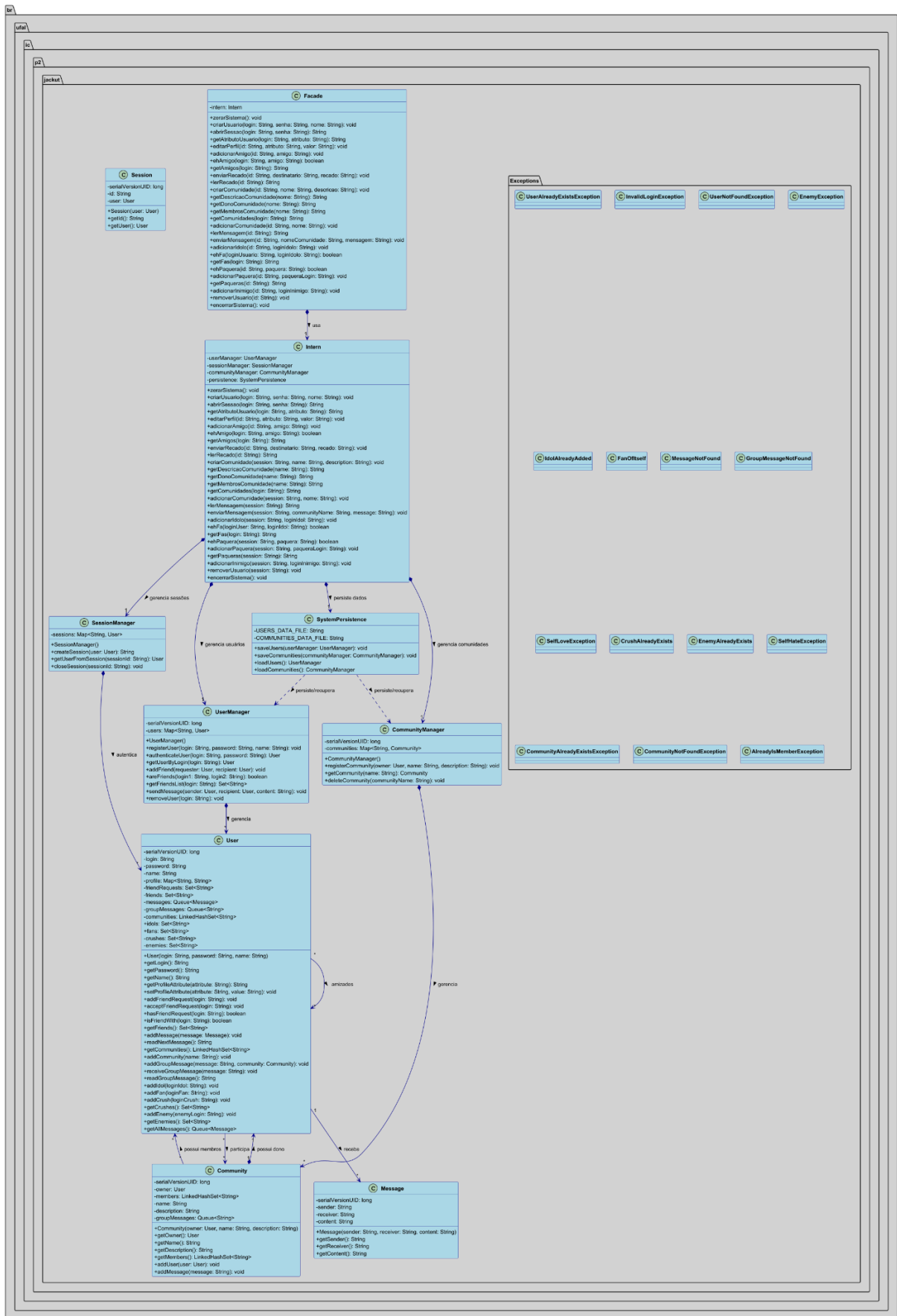


diagrama de classes do projeto.

Princípios de Orientação a Objetos

O design do *Jackut* foi fundamentado em princípios de orientação a objetos para garantir clareza, manutenibilidade e flexibilidade. O encapsulamento é rigorosamente aplicado, com todas as classes utilizando atributos privados e métodos públicos para acesso controlado. Por exemplo, a classe *User* protege dados sensíveis, como credenciais de login e listas de amigos, expondo-os apenas por meio de métodos como `getLogin()` e `addFriend()`. Herança e polimorfismo são utilizados para aumentar a extensibilidade. Exceções personalizadas, como *UserNotFoundException*, derivam da classe *RuntimeException*, enquanto a classe *Intern* delega operações a gerenciadores especializados, permitindo comportamento polimórfico. A composição e a agregação também fortalecem o design. A classe *User* compõe objetos como *Message* e *Community*, enquanto a classe *CommunityManager* agrega instâncias de *Community*, que podem existir de forma independente.

Padrões de Projeto

Diversos padrões de projeto foram incorporados para aprimorar a arquitetura do sistema. O padrão **Facade**, implementado pela classe *Facade*, simplifica a interação com o sistema ao ocultar a complexidade das operações internas, sendo especialmente útil para testes com ferramentas como *EasyAccept*. O padrão **Singleton** é aplicado implicitamente nas classes *UserManager* e *CommunityManager*, que são instanciadas uma única vez na classe *Intern*, garantindo um ponto centralizado de acesso aos dados. Além disso, o padrão **Command** é utilizado para gerenciar filas de mensagens nas classes *User* e *Community*, permitindo o processamento sequencial das comunicações.

Escolhas de Design

O sistema reflete várias decisões de design cuidadosamente planejadas para garantir robustez e usabilidade. Exceções personalizadas, como *UserNotFoundException* e *CommunityAlreadyExistsException*, foram criadas para fornecer mensagens de erro claras e específicas, facilitando a depuração. Por exemplo, o método `getUserByLogin` lança uma *UserNotFoundException* caso o usuário não seja encontrado, melhorando a legibilidade do código. A persistência de dados é realizada por meio de serialização, com a classe *SystemPersistence* salvando e carregando dados em arquivos binários através de métodos como `saveUsers()` e `loadUsers()`, garantindo que o estado do sistema seja preservado entre execuções. As responsabilidades são bem separadas, com a classe *Intern* centralizando a lógica e delegando tarefas a gerenciadores como *UserManager*, para operações de usuário, e *CommunityManager*, para gerenciamento de comunidades. As relações sociais, incluindo amizades, ídolos e interesses românticos, são implementadas usando conjuntos na classe *User*, com métodos

como `addFriend()` e `isFriendWith()` que validam interações e evitam erros, como tentativas de auto-relação.

Pontos Fortes e Melhorias

O design do *Jackut* apresenta diversos pontos fortes. A separação clara de responsabilidades resulta em baixo acoplamento entre as classes, facilitando a manutenção. A arquitetura extensível permite a adição de novas funcionalidades, como novos tipos de relações sociais, sem alterar o código existente. Além disso, o uso de exceções personalizadas melhora a clareza no tratamento de erros. Para aprimoramentos futuros, seria interessante implementar o padrão Observer para notificar usuários sobre novas mensagens ou atualizações em tempo real. A substituição da serialização por formatos como JSON ou XML poderia facilitar a leitura e manipulação dos dados. Por fim, a inclusão de um sistema de logs para registrar atividades do sistema poderia melhorar o monitoramento e a depuração.

Conclusão

O projeto *Jackut* foi desenvolvido com base em boas práticas de orientação a objetos, promovendo encapsulamento, baixo acoplamento e alta coesão. A adoção do padrão Facade simplifica a interação com o sistema, enquanto os gerenciadores centralizam a lógica de negócio de forma eficiente. Com um design extensível e de fácil manutenção, o sistema atende plenamente aos requisitos da disciplina, demonstrando a aplicação prática dos conceitos de programação orientada a objetos.